

App Dev II

Project Report

Placement Portal Application - V2

Jan 2026 Term

1. Student Details

Name: Ashitosh Jagtap
Roll Number: 23f3000434
Email: 23f3000434@ds.study.iitm.ac.in

2. Project Details

Project Title: Placement Portal Application - V2

Problem Statement:

Institutes require efficient systems to manage campus recruitment activities involving companies and students. Currently, many institutes rely on spreadsheets, emails, or manual coordination, which makes it difficult to manage company approvals, placement drives, student registrations, and application tracking. The goal is to build a Placement Portal Application that allows Admin (Institute), Companies, and Students to interact with the system based on their roles.

Approach:

I built the application using a modular Flask backend with role-based JWT authentication (Admin, Company, Student). The database uses a unified User model linked to separate StudentProfile and CompanyProfile tables. PlacementDrive and Application models handle the core recruitment workflow with state machines (Pending > Approved > Closed for drives; Applied > Shortlisted > Selected/Rejected for applications). The frontend uses Vue.js 3 via CDN with Vue Router for SPA navigation and Bootstrap 5 for styling. Redis handles caching with configurable TTLs, and Celery manages three background jobs: daily deadline reminders, monthly HTML activity reports via email, and user-triggered CSV exports.

3. AI/LLM Declaration

I used ChatGPT to assist with minor debugging tasks and resolving specific error messages during development. The extent of AI/LLM usage is around 5-10%, limited to:

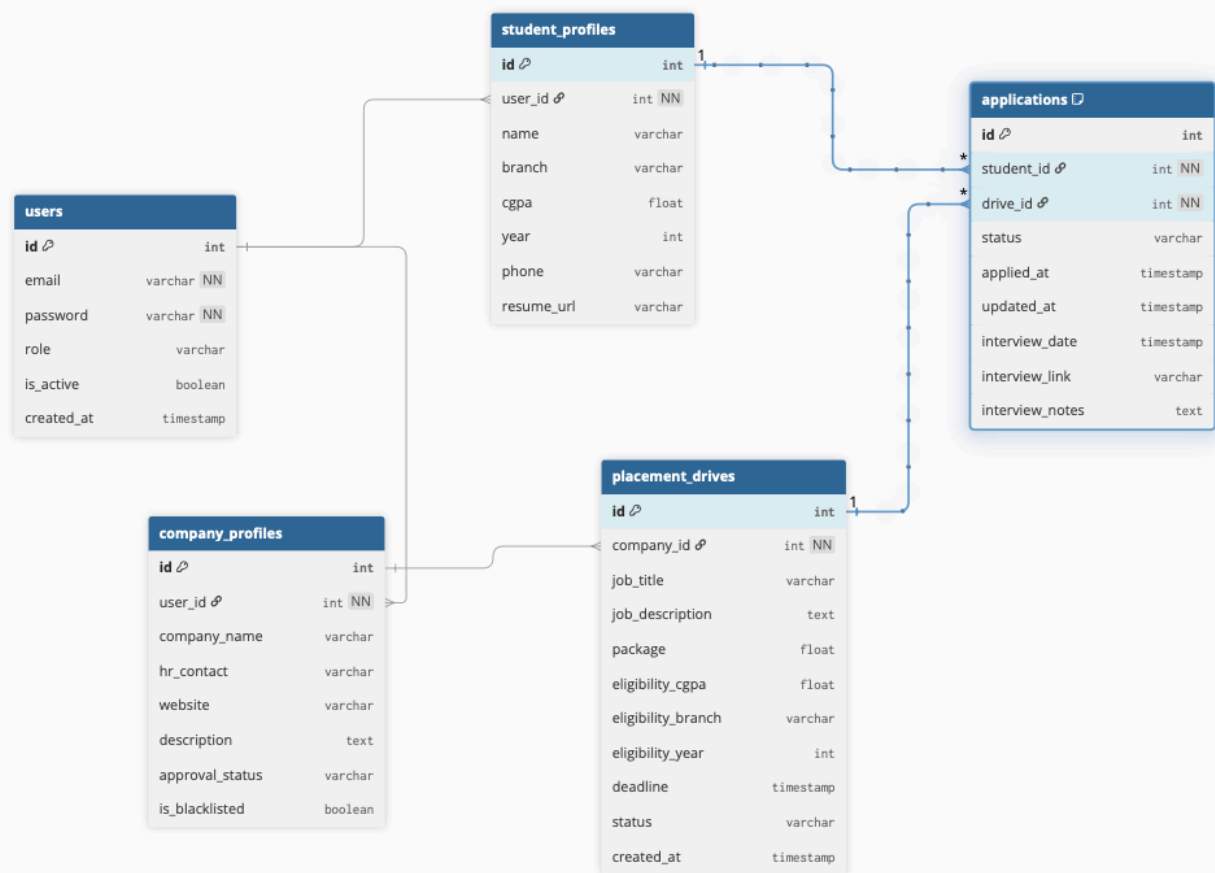
- * Debugging specific Flask/SQLAlchemy error messages
- * Clarifying Vue.js reactivity behavior for edge cases
- * Minor syntax reference for Celery task configuration

All application architecture, database design, business logic implementation, frontend UI development, API design, testing, and integration were done entirely by me manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask 3.1.0	Core backend web framework and API server
Flask-SQLAlchemy 3.1.1	ORM for SQLite database operations
Flask-JWT-Extended 4.7.1	JWT-based authentication and authorization
Flask-CORS 5.0.1	Cross-Origin Resource Sharing
Flask-Caching 2.3.1	Redis/SimpleCache caching layer
Flask-Mail 0.10.0	Email sending (SMTP via Gmail)
bcrypt 4.3.0	Password hashing and verification
Celery 5.4.0	Async background task processing
Redis 5.2.1	Cache backend and Celery message broker
SQLite	Lightweight relational database
Vue.js 3.4 (CDN)	Reactive frontend UI framework
Vue Router 4.3 (CDN)	Client-side SPA routing
Bootstrap 5.3.3 (CDN)	Responsive UI styling and components
Chart.js 4.4 (CDN)	Data visualization (charts/graphs)
Jinja2	Template engine (entry point only)

5. Database Schema / ER Diagram



Relationships:

- * One-to-One: users > student_profiles (via user_id)
- * One-to-One: users > company_profiles (via user_id)
- * One-to-Many: company_profiles > placement_drives (via company_id)
- * One-to-Many: student_profiles > applications (via student_id)
- * One-to-Many: placement_drives > applications (via drive_id)

6. API Resource Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register student or company
POST	/api/auth/login	Authenticate and get JWT
GET	/api/admin/dashboard	Admin stats overview
GET	/api/admin/companies	List companies (search/filter)
PUT	/api/admin/companies/<id>/approve	Approve company
PUT	/api/admin/companies/<id>/reject	Reject company
PUT	/api/admin/companies/<id>/blacklist	Toggle blacklist
GET	/api/admin/drives	List all drives
PUT	/api/admin/drives/<id>/approve	Approve drive
PUT	/api/admin/drives/<id>/reject	Reject drive
PUT	/api/admin/drives/<id>/close	Close drive
GET	/api/admin/students	List students (search)
GET	/api/admin/applications	View all applications
PUT	/api/admin/users/<id>/toggle-active	Toggle user active
GET	/api/admin/stats	Statistics (cached)
GET	/api/company/dashboard	Company dashboard
GET,PUT	/api/company/profile	Get/update profile
GET,POST	/api/company/drives	List/create drives
GET	/api/company/drives/<id>/apps	View drive applications
PUT	/api/company/apps/<id>/status	Update app status
PUT	/api/company/apps/<id>/interview	Schedule interview
GET	/api/student/dashboard	Student dashboard
GET,PUT	/api/student/profile	Get/update profile
POST	/api/student/resume-upload	Upload resume (PDF/DOC)
GET	/api/student/drives	Browse drives (search/filter)
POST	/api/student/drives/<id>/apply	Apply to drive
GET	/api/student/applications	Application history
POST	/api/student/export	Export CSV (async Celery)

7. Video Presentation

Drive Link:

https://drive.google.com/file/d/1Cp_iTOKZxjmp-nhgucELd0dg-jDtahTS/view?usp=share_link

(Accessible to anyone with the link, View permission enabled)